

Assignment: Parallelize an Image Blur Algorithm (Gaussian Blur)

Dr. Xiaoyuan Suo (xiaoyuansuo51@webster.edu)

Learning goals

By the end, you will be able to:

- Implement a correct parallel image-processing pipeline using multiprocessing.
- Handle boundary (halo) regions correctly when splitting images.
- Measure and visualize performance: runtime, speedup, efficiency, and overhead.
- Explain scaling limits (Amdahl's Law, process overhead, memory bandwidth, cache effects).

Your Task

You are given one or more images. Implement two versions of Gaussian blur:

1. Serial blur: `blur_serial(img, ksize, sigma)`
2. Parallel blur: `blur_parallel(img, ksize, sigma, workers, method=...)`

Your parallel implementation must:

- Split the image into chunks
- Blur chunks in parallel
- Reassemble into a final image
- Produce output that matches the serial result (within specified tolerance)

Required blur kernel

Use OpenCV Gaussian blur:

- `cv2.GaussianBlur(img, (ksize, ksize), sigma)`
- Use odd $ksize \geq 3$ (default for experiments: $ksize=15$, $\sigma=0$)

Specific requirements

A. Must handle boundary correctness (halo requirement)

If you split an image into horizontal stripes and blur each stripe independently, pixels near the stripe border will be wrong unless you include a halo (extra rows) from neighboring stripes.

You must implement halo handling:

- Let $r = ksize // 2$ be the blur radius.
- When worker processes a stripe `[start:end]`, it must receive an expanded stripe:
 - `[start-r : end+r]` clipped to image bounds
- After blurring, you must crop back to `[start:end]` before reassembly.

Deliverable must include a correctness test that detects border artifacts if halo is missing.

B. Required parallel method

Use Python multiprocessing:

- multiprocessing.Pool or concurrent.futures.ProcessPoolExecutor
- Threads-only solutions are not allowed (unless you justify why threads scale in OpenCV on your platform—rare; must show evidence).

C. No “one-liner” solutions

Not allowed as your main implementation:

- Calling cv2.GaussianBlur once on the whole image and claiming it is parallel.
- Using OpenCV internal threading as “parallelization” without your own partitioning.

D. Reproducible experiments

You must provide a script that runs benchmarks automatically and writes raw results to CSV/JSON.

Input images (provided or generated)

You will test on at least 3 images (two are provided on Canvas, you should use an image of yourself as the third):

1. Small: ~1–2 MP (e.g., 1280×720)
2. Medium: ~8 MP (e.g., 3840×2160 / 4K)
3. Large: ~24–48 MP (e.g., 8000×6000 or similar)

Deliverables

1) Code package

Submit a zip or repo containing:

- blur.py (core functions)
- run_experiments.py (benchmark runner)
- readme.md (explanations)
- results/ folder:
 - raw timing logs (.csv or .json)
 - generated plots (.png)
 - output images (serial_*.png, parallel_*.png, diff_*.png)

2) Short report (1 page max, PDF)

Include:

(i) Correctness evidence (required)

For each test image:

- Save serial output and parallel output
- Compute and report:
 - max absolute pixel difference
 - mean absolute pixel difference
- Create and save a difference image (visual artifact):

- $\text{diff} = \text{abs}(\text{serial} - \text{parallel})$ and scale to visible range
- Provide at least one failing example showing what happens if halo is omitted (you can include a “no_halo” mode for demonstration).

Correctness rule:

Parallel output must match serial output with:

- $\text{max_diff} \leq 1$ for uint8 images (due to rounding)
OR justify a different tolerance with evidence.

(ii) Performance plots (required)

For each image size, plot:

- Runtime vs workers
- Speedup vs workers
- Efficiency vs workers

(iii) Explanation

Answer briefly (bullet points/ half sentences ok):

1. What overhead exists in your multiprocessing design? (process startup, IPC/pickling, recombine, cache misses, etc.)
2. Why does speedup saturate?
3. How did you choose chunk size / stripe count? Why horizontal stripes vs vertical?
4. Why does halo size equal $\text{ksize}/2$?

Experimental requirements

Worker sweep

Test worker counts:

- 1, 2, 4, 8, ... up to:
 - physical cores OR
 - no further improvement

Trial policy

- Run 3 trials per configuration
- Report median runtime

Control hidden threading

To avoid OpenCV/NumPy using their own threads (which hides effects), set environment variables in your benchmark script:

- `OMP_NUM_THREADS=1`
- `MKL_NUM_THREADS=1`
- `OPENBLAS_NUM_THREADS=1`

Include these in your README.

Required program interface

Implement these functions:

`blur_serial(img, ksize=15, sigma=0) -> blurred_img`

`blur_parallel(img, ksize=15, sigma=0, workers=4) -> blurred_img`

- must use halo handling

`compute_diff_metrics(img_a, img_b) -> (max_diff, mean_diff)`

`save_diff_image(img_a, img_b, path)`

Benchmark script should support:

`python run_experiments.py --images img1.jpg img2.jpg img3.jpg --ksize 15 --sigma 0 --workers 1 2 4 8`

Live check (5 minutes, in lab)

You will:

- Run your benchmark script on one image with workers 1/2/4
- Show your diff metrics and diff image
- Point to the exact code section where halo is added and cropped

Grading rubric (100 pts)

Correctness (30)

- Matches serial within tolerance (15)
- Diff metrics + diff images saved (10)
- Demonstrates halo necessity with failing “no halo” run (5)

Parallel design (25)

- Proper chunking strategy (10)
- Halo implementation correct (10)
- Clean recombination and minimal copying (5)

Performance methodology (25)

- Worker sweep + 3 trials + median (10)
- Plots (runtime/speedup/efficiency) for 3 sizes (10)
- Reproducible scripts + raw logs (5)

Analysis & explanation (20)

- Overhead + saturation explained using results (10)
- Chunking and halo reasoning clear (10)